

DA4: Reshaping data

First let's load the requisite packages and some data (surveys).

```
In [1]: # A Python data analysis and manipulation tool
import pandas as pd

# Python equivalent of `ggplot2`
from plotnine import *

# If using seaborn for plotting
import seaborn as sns
import matplotlib.pyplot as plt

surveys_join = pd.read_csv("data/surveys_join.csv")
```

*Now we use the **.pivot()** method to pivot wider*

wide

id	x	y	z
1	a	c	e
2	b	d	f

```
In [2]: surveys_wide = surveys_join.pivot(  
        index = "record_id",  
        columns = "year",  
        values = "weight"  
    )  
  
surveys_wide.head()
```

Out [2]:

	year	2021	2022	2023	2024
record_id					
118	199.5	192.8	192.6	187.6	
160	192.5	189.0	184.3	181.4	
166	195.4	189.5	183.6	182.0	
178	190.1	188.4	182.2	177.5	
184	195.1	186.7	193.5	187.3	

Let's tidy up those headers with a `yr|` prefix (unfortunately this will break `record_id` so there's an extra step to fix it).`_`

```
In [3]: # Add 'yr_' prefix to year columns
surveys_wide.columns = [f"yr_{col}" for col in surveys_wide.columns]

surveys_wide.head()
```

Out [3]:

	yr_2021	yr_2022	yr_2023	yr_2024
record_id				
118	199.5	192.8	192.6	187.6
160	192.5	189.0	184.3	181.4
166	195.4	189.5	183.6	182.0
178	190.1	188.4	182.2	177.5
184	195.1	186.7	193.5	187.3

```
In [4]: # Reset index to make 'record_id' a column again
surveys_wide = surveys_wide.reset_index()

surveys_wide.head()
```

Out [4]:

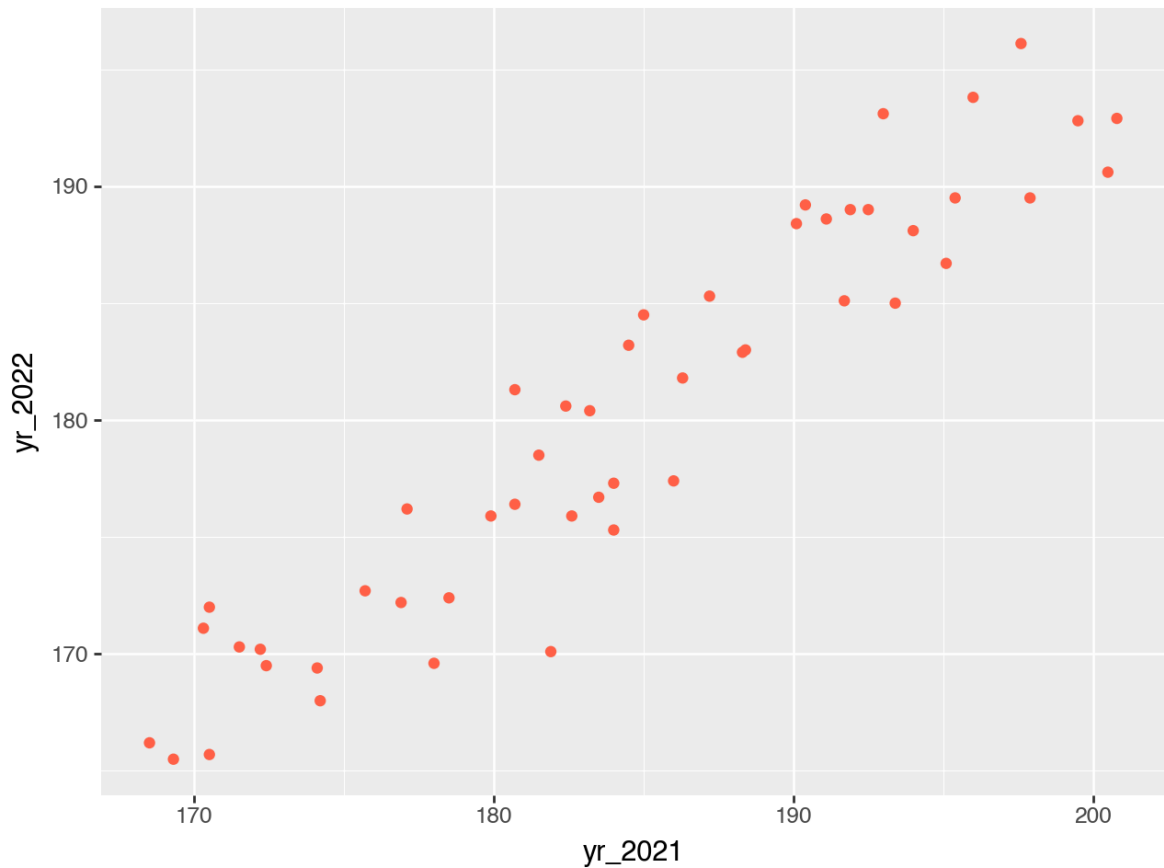
	record_id	yr_2021	yr_2022	yr_2023	yr_2024
0	118	199.5	192.8	192.6	187.6
1	160	192.5	189.0	184.3	181.4
2	166	195.4	189.5	183.6	182.0
3	178	190.1	188.4	182.2	177.5
4	184	195.1	186.7	193.5	187.3

Example usage: Explore any possible relationship between 2021 and 2022 using plotnine aka why did we do all of this processing?

```
In [5]: # Colour the points tomato red - because why not? Demonstrates setting pa

p = (ggplot(surveys_wide, aes(x = "yr_2021", y = "yr_2022"))) +
    geom_point(colour="tomato")
```

```
p.show()
```



For reasons unknown, the opposite to **.pivot()** is **.melt()**. In the interests of moving forward, let's just use it.

```
In [7]: surveys_long = surveys_wide.melt(
        id_vars = "record_id",          # columns to keep fixed
        var_name = "year",              # name of the new 'year' column
        value_name = "weight"          # name of the new 'weight' column
    )

surveys_long.head()
```

```
Out [7]:
```

	record_id	year	weight
0	118	yr_2021	199.5
1	160	yr_2021	192.5
2	166	yr_2021	195.4
3	178	yr_2021	190.1
4	184	yr_2021	195.1

Let's fix those year column values back to numbers by substituting the string and then casting to integer.

```
In [8]: surveys_long["year"] = surveys_long["year"].str.replace("yr_", "").astype(int)

surveys_long.head()
```

Out [8]:

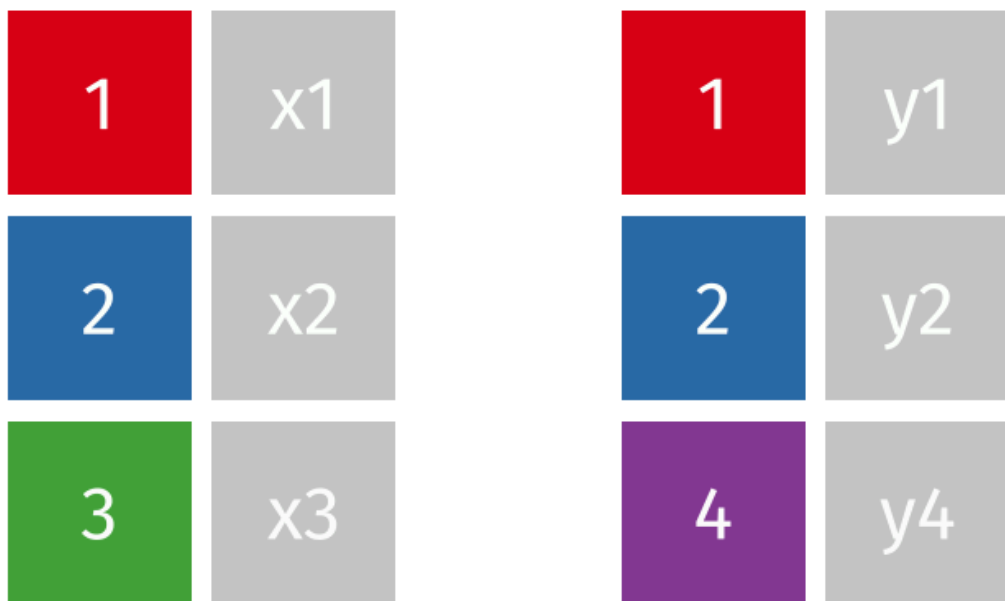
	record_id	year	weight
0	118	2021	199.5
1	160	2021	192.5
2	166	2021	195.4
3	178	2021	190.1
4	184	2021	195.1

Much better !

DA4: Combining data

Different types of joins

`inner_join(x, y)`



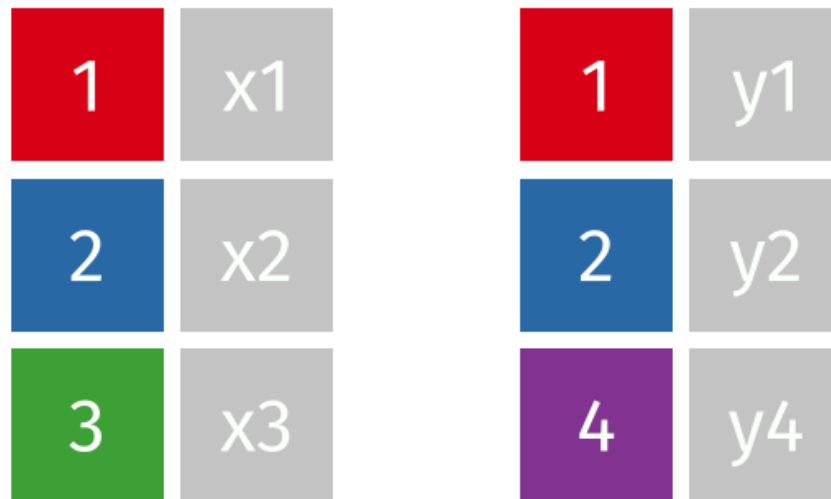
left_join(x, y)

1	x1		1	y1
2	x2		2	y2
3	x3		4	y4

right_join(x, y)

1	x1		1	y1
2	x2		2	y2
3	x3		4	y4

full_join(x, y)



Let's do a simple Left join to illustrate how to do this in Python

Then when you work through the chapter you can try other joins.

```
In [15]: # A Python data analysis and manipulation tool
import pandas as pd

# Python equivalent of `ggplot2`
from plotnine import *

surveys = pd.read_csv("data/surveys.csv")

# Read in a file that can be used to translate plotIDs into plot-types
plot_types = pd.read_csv("data/plots.csv")

plot_types # list the plot types
```

```
Out[15]:
```

	plot_id	plot_type
0	1	Spectab exclosure
1	2	Control
2	3	Long-term Krat Exclosure
3	4	Rodent Exclosure
4	5	Short-term Krat Exclosure

We use the **.merge()** method with data coming from **surveys** and **plot_types**, the how being a **left-join** and the join being on the common field of **plot_id**.

```
In [16]: surveys_left = pd.merge(surveys, plot_types, how = "left", on = "plot_id")
surveys_left
```

```
Out[16]:
```

	record_id	month	day	year	plot_id	species_id	sex	hindfoot_length	v
0	1	7	16	1977	2	NL	M	32.0	
1	2	7	16	1977	3	NL	M	33.0	
2	3	7	16	1977	2	DM	F	37.0	
3	4	7	16	1977	7	DM	M	36.0	
4	5	7	16	1977	3	DM	M	35.0	
...	...	...	...	...	...	...	...	...	
35544	35545	12	31	2002	15	AH	NaN	NaN	
35545	35546	12	31	2002	15	AH	NaN	NaN	
35546	35547	12	31	2002	10	RM	F	15.0	
35547	35548	12	31	2002	7	DO	M	36.0	
35548	35549	12	31	2002	5	NaN	NaN	NaN	

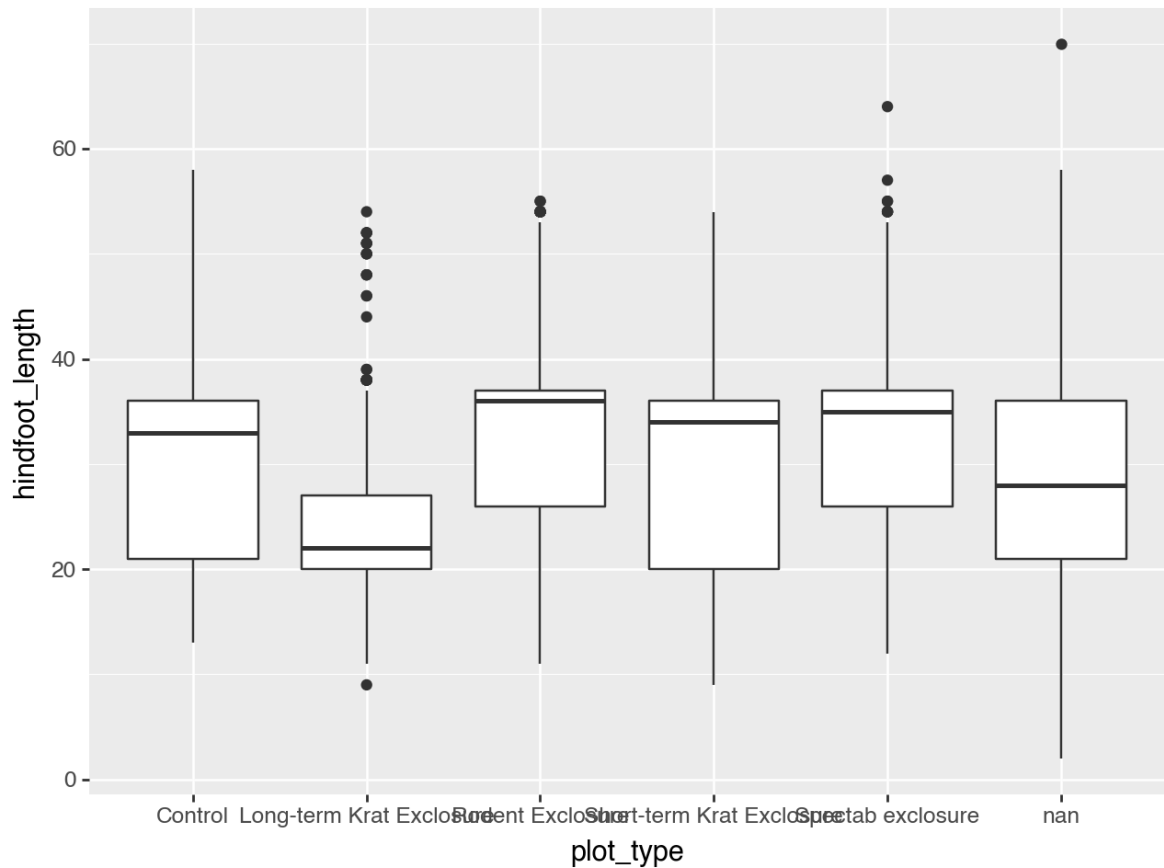
35549 rows × 10 columns

Let's use our newly derived data structure to create a box-plot of `plot_type` vs. `hindfoot_length` that names the type of enclosure rather than using an id number.

```
In [9]: p = (ggplot(surveys_left, aes(x = "plot_type", y = "hindfoot_length")) +
geom_boxplot())

p.show()
```

/Users/fernan02/miniforge3/lib/python3.12/site-packages/plotnine/layer.py: 293: PlotnineWarning: stat_boxplot : Removed 4111 rows containing non-finite values.



Now work through the material of chapter 13 and try the exercises at the end.

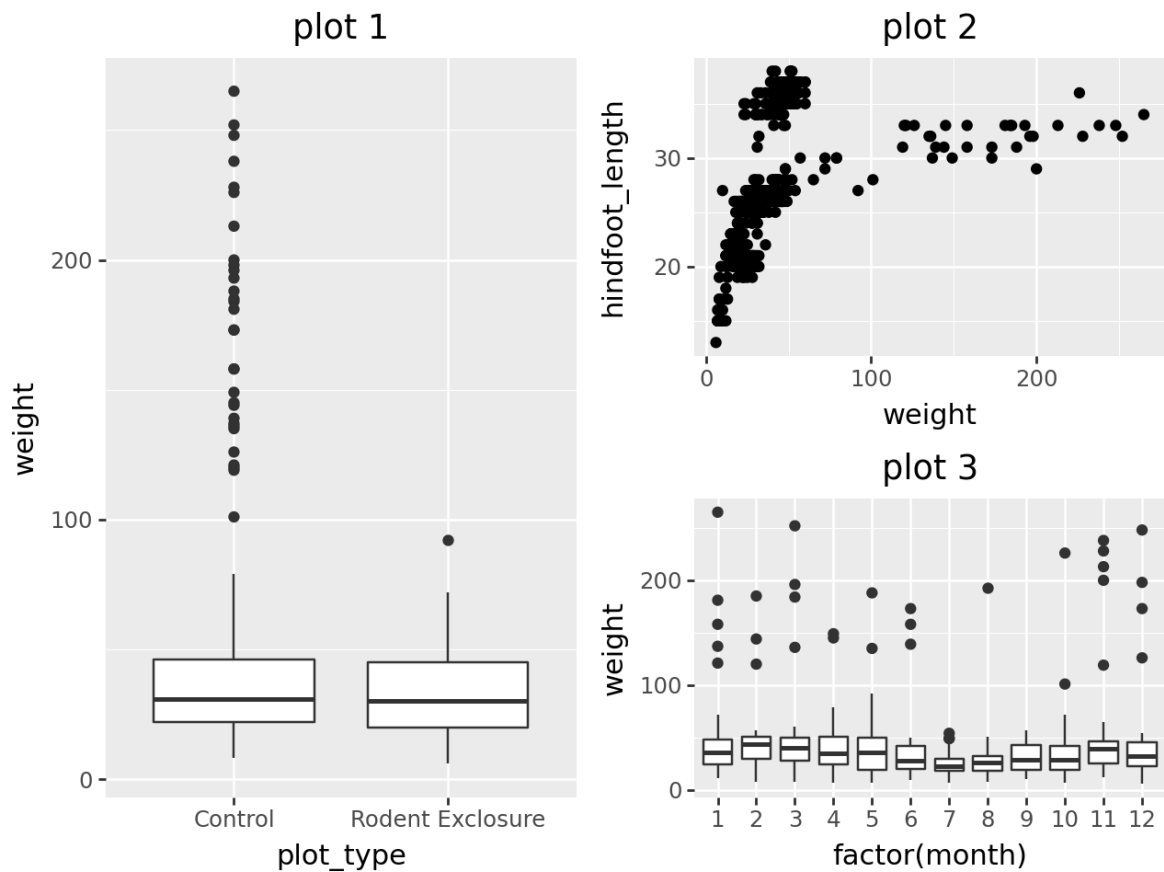
```
In [17]: surveys_y2k = (
    surveys_left[
        surveys_left["plot_type"].isin(["Control", "Rodent Exlosure"])
        & surveys_left["year"].isin([2000, 2001, 2002])]
    .dropna())
```

```
In [18]: p1 = (ggplot(surveys_y2k, aes(x='plot_type', y='weight')) +
    geom_boxplot() +
    labs(title="plot 1"))

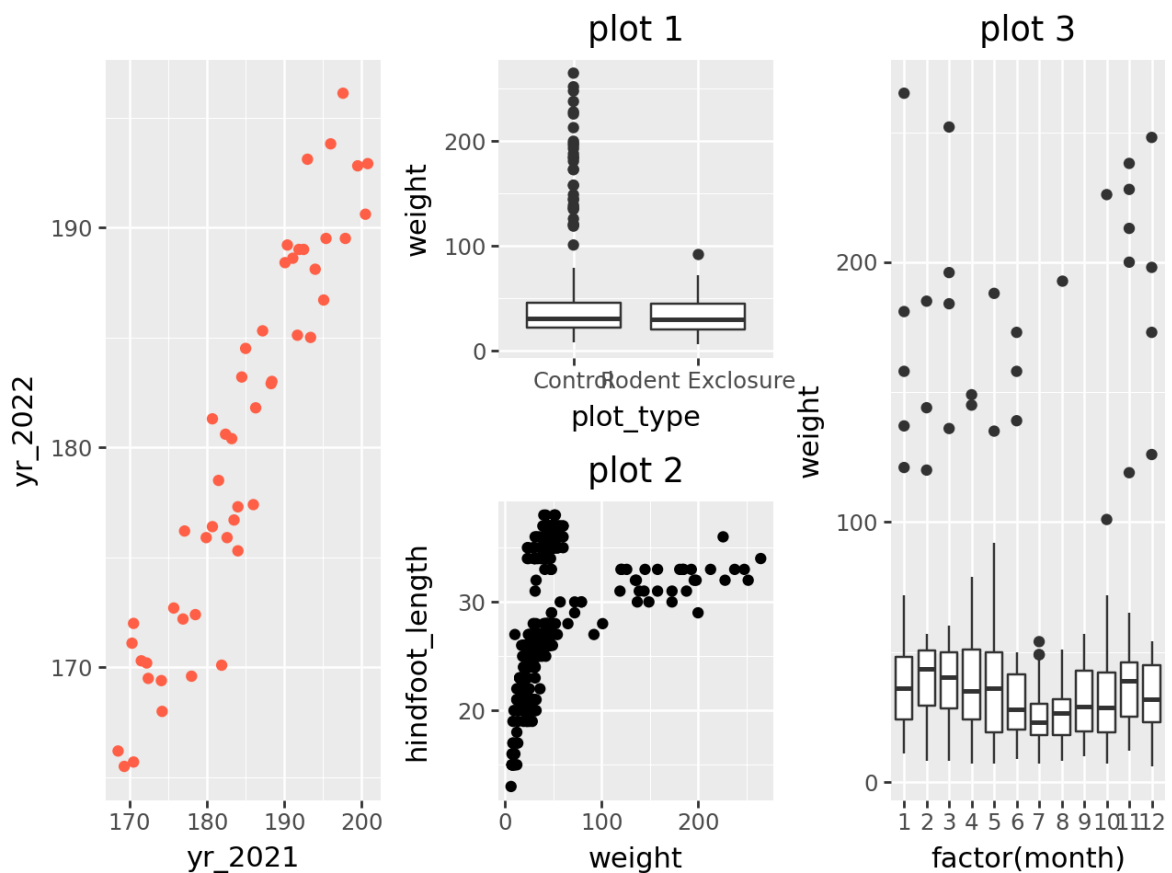
p2 = (ggplot(surveys_y2k, aes(x='weight', y='hindfoot_length')) +
    geom_point() +
    labs(title="plot 2"))

p3 = (ggplot(surveys_y2k, aes(x='factor(month)', y='weight')) +
    geom_boxplot() +
    labs(title="plot 3"))

p1 | p2 / p3
```



In [22]: p | p1 / p2 | p3



```
In [16]: import webbrowser
# Open URL in the default browser to Plotnine Gallery
webbrowser.open("https://plotnine.org/gallery/", new=2) # Opens in a new
```

Out[16]: True

In []: